IREE / MLIR 结构化 IR

图级 cim-fuse-heterogeneous-contractions

矩阵收缩

`linalg.matmul`
`linalg.generic` contraction

逐元素计算

`linalg.generic`
`arith.addi / subi / muli / divsi`

复制与清零

identity `linalg.generic`
`linalg.fill(0)`

外部张量与静态权重

`hal.interface.binding.subspan`
`memref.global`

`cim-select-lowering-strategy`
`cim-legalize-array-tiles`
IREE bufferization
`cim-convert-linalg-to-cim`

IREE bufferization
`cim-convert-linalg-to-cim`

`cim-convert-linalg-to-cim`
identity → copy
`cim-convert-to-target-ir`
fill → copy

原生 HAL binding / workgroup
保留到 serializer

逻辑 CIM IR

`cim.mvm``(src, weight, dst)`
覆盖写；仍可重排权重

`cim.elt`
add / sub / mul / div · vv / vi

`memref.copy`
数据搬运仍使用标准 MLIR op

HAL subspan / native views
memref.global · 静态权重

链接与物理化

cim-link-executables → cim-pack-horizontal-mvms cim-replicate-weight-mvms → cim-plan-array → cim-materialize-sram → cim-expand-bit-serial → cim-allocate-sram reuse=true / false

固定坐标与地址

物理 CIM IR

`cim.bit`
+ `cim.elt` 结果重建

`cim.elt`
操作码与 vv / vi 模式保持不变

memref.copy
binding / embedded / SRAM view

静态权重 → **`cim.array.load`**
动态缓冲区 → fixed `memref.view`

CIMExecutablePayload 读取物理 op 与 memref 视图

ISA 指令

`cim.bit.i8.i8`
`cim.bit.i32.i8`
`elt.mul / add / sub`

`elt.{op}.{dtype}.{vv|vi}`

`mem.copy.{dst}.{src}`

`cim.array.load.i8` · `row, bit_col, K, N`
memref views → 其他指令的地址操作数